

Anexo de Documentación de Prompts

1. Generación de Empresa y Dataset

Campo	Descripción
N°	P-01
Objetivo	Definir empresa ficticia , rubro y problema de negocio
Herramienta	Claude Sonnet 5(medium)
Texto del prompt	Necesito definir una empresa ficticia de telecomunicaciones para un proyecto de Machine Learning. Debe permitir un problema de regresión (predecir un valor continuo, como el valor de vida del cliente) y un problema de clasificación binaria (predecir abandono/churn). Ayúdame a definir el nombre, rubro, mercado, portafolio de productos y el problema de negocio, con el nivel de detalle suficiente para que después pueda justificar cada variable de un dataset con una razón de negocio explícita
Resultado / cómo se usó	Se definió a NovaConecta, operador de telecomunicaciones (internet fijo/móvil y telefonía) con 6 años en el mercado peruano, tres planes (Básico, Estándar, Premium) con distinta participación de mercado, y el problema de negocio doble: estimar el Valor de Vida del Cliente (CLV) y predecir el abandono (churn) sobre la misma cartera de 1,200 clientes. Este contexto se usó como base para diseñar el esquema de variables y, posteriormente, el script generador del dataset.
Validación / ajuste del equipo	Se revisó que la definición cubriera ambos tipos de problema exigidos (regresión y clasificación) sobre el mismo caso de negocio, y que el rubro elegido (telecomunicaciones) permitiera justificar de forma natural variables de comportamiento (plan, consumo, soporte, satisfacción) necesarias para los Informes 3 a 6. No se requirieron ajustes adicionales: la definición se mantuvo estable durante todo el proyecto.

Campo	Descripción
N°	P-02
Objetivo	Generar el dataset sintético de la empresa ficticia para los Informes 3 a 6
Herramienta	Claude Sonnet 5(medium)

Texto del prompt	Genera código Python (numpy, pandas, Faker) para crear un dataset sintético de 1200 clientes de la empresa ficticia 'NovaConecta' (telecomunicaciones). Usa Faker en español para los datos demográficos. Variables y relaciones: plan_contratado (Básico/Estándar/Premium, p=45/35/20%), antigüedad_meses (coherente con una fecha de registro aleatoria en los últimos 5 años), uso_datos_gb (normal, dependiente del plan contratado), cargo_mensual (tarifa base del plan + consumo de datos + ruido), num_llamadas_soporte (Poisson), satisfaccion_cliente (escala 1-10, penalizada por soporte y cargos altos), abandono (Si/No, variable binaria dependiente de satisfacción, cargo mensual y antigüedad mediante función logística) y valor_vida_cliente (CLV continuo, calculado a partir del cargo mensual, antigüedad y estado de abandono). Introduce correlaciones de negocio realistas, un pequeño porcentaje de nulos (en uso_datos_gb) y outliers (en cargo_mensual) para hacerlo útil para un EDA. Fija semilla=42 para reproducibilidad. Exporta a un archivo CSV llamado 'dataset_novaconecta.csv' guardándolo de manera portable en el mismo directorio del script utilizando la librería os.
Resultado / cómo se usó	Se obtuvo el script generador (generar_dataset.py); el equipo lo ejecutó, revisó las distribuciones resultantes por plan y ajustó los rangos y parámetros (medias, desviaciones, coeficientes de la función logística) hasta que reflejaran de forma realista el comportamiento esperado de la empresa.
Validación / ajuste del equipo	Se verificó la coherencia de las correlaciones (uso de datos vs. cargo mensual, llamadas a soporte vs. satisfacción) mediante un análisis exploratorio preliminar, se confirmó que la antigüedad calzara exactamente con la fecha de registro, y se corrigió el desbalance de la variable objetivo abandono ajustando el ruido de la función logística hasta obtener una proporción realista (~62% No / ~38% Sí).

2. Informe 1

Campo	Descripción
Nº	P-03
Objetivo	Estructurar el modelado del 8-Puzzle, generar el código base en Java de los algoritmos de búsqueda ciega (BFS, DFS) e informada (Greedy, A*), e implementar un recolector de métricas de desempeño.

Herramienta	Claude Sonnet5
Texto del prompt	Actúa como un Ingeniero de Software experto en Java e Inteligencia Artificial. Necesito implementar el problema del 8-Puzzle modelado como un árbol de búsqueda en el espacio de estados, ejecutable por consola (CLI) con Java (SDK 11+). La solución debe estar estructurada en el paquete por defecto (sin declarar "package") para que pueda compilarse directamente desde la carpeta `src` usando `javac Main.java` y `java Main`. Divide la lógica en las siguientes 4 clases independientes y limpias: - State.java (Modelo del Tablero y Reglas) * Modela el estado como un arreglo unidimensional de 9 posiciones (o matriz 3x3) donde el `0` representa el espacio vacío. * Implementa el Estado Meta: `[1, 2, 3, 4, 5, 6, 7, 8, 0]`. * Implementa la lógica de transición `getSuccessors()`: Acciones posibles (Mover el espacio en blanco hacia Arriba, Abajo, Izquierda, Derecha). Costo de cada acción: `1`. * Implementa un renderizador ANSI (`printState()`): * Dibuja una cuadrícula limpia. * Fichas numeradas en color Cian (`\u001B[36m`). * Espacio vacío representado por un bloque gris u oscuro o simplemente en negro (`\u001B[90m`). - SearchNode.java (Estructura del Árbol de Búsqueda) * Clase para envolver los estados en nodos del árbol. Debe contener: * Referencia al `State` actual. * Referencia al `SearchNode` padre (para reconstruir la ruta). * String con la `action` que generó este nodo. * `gCost`: Costo del camino desde la raíz (profundidad). * `hCost`: Valor de la heurística. * Implementa un método `getF()` que retorne `gCost + hCost`. - SearchAlgorithms.java (Motor de Inteligencia Artificial) * Implementa los 4 algoritmos requeridos usando las estructuras de datos clásicas: - BFS (Búsqueda a lo Ancho) usando `Queue`. - DFS (Búsqueda en Profundidad) usando `Stack` (con control estricto de estados visitados para evitar ciclos infinitos). - Greedy Best-First Search usando `PriorityQueue` ordenada por `hCost`.

	4. A* usando `PriorityQueue` ordenada por `fCost = gCost + hCost`. * Heurística: Implementa la Distancia de Manhattan como función heurística $h(n)$ para Greedy y A*. * Métricas: Cada método debe llevar un contador local de `nodosExpandidos`. El método debe retornar un objeto/estructura con: el nodo meta encontrado (o null), la cantidad de nodos expandidos, y el costo de la solución. 4. Main.java (Controlador y CLI) * Muestra un menú de bienvenida ANSI y permite al usuario elegir entre casos de prueba predefinidos (Fácil: solución a 5 pasos; Medio: solución a 14 pasos; Difícil: solución a 22 pasos) o ingresar un tablero manualmente validando que tenga solución (inversiones pares). * Al ejecutar un caso, el sistema debe correr los 4 algoritmos secuencialmente sobre el mismo estado inicial. * Recolecta el Tiempo de Ejecución de cada algoritmo usando `System.nanoTime()` y conviértelo a milisegundos. * Reporte Comparativo: Al finalizar, limpia la pantalla (`\033[H\033[2J` y flush) e imprime una Tabla Comparativa estilizada en consola (usando bordes ASCII/ANSI) que muestre lado a lado para cada algoritmo: * Nombre del algoritmo. * Nodos Expandidos. * Costo de la solución (longitud de la ruta). * Tiempo de ejecución (ms). * Muestra la secuencia de movimientos paso a paso de la solución óptima obtenida por A*. Requisitos Técnicos de Codificación: * Manejo estricto de conjuntos de "Visitados" (Closed Set) usando `HashSet` de los estados para optimizar memoria y tiempo, asegurándote de sobrescribir `equals()` y `hashCode()` en la clase `State`.
Resultado / cómo se usó	Se obtuvo el código fuente funcional dividido en 4 clases (State, SearchNode, SearchAlgorithms, Main). El código base se integró directamente en la carpeta src/ del informe_1_busqueda del repositorio. El sistema renderiza el tablero interactivo y compila con éxito una tabla que cruza los nodos expandidos, costo y tiempo de los 4 algoritmos solicitados.

Validación / ajuste del equipo	El equipo de Búsqueda (Java) validó matemáticamente la heurística de Manhattan implementada en el código, verificando que fuera admisible y consistente. Se ajustó la configuración de casos de prueba del Main.java para evitar desbordamientos de memoria en DFS al enfrentar estados con profundidad extrema, acotando la profundidad óptima en la selección de casos predefinidos y asegurando el control de ciclos mediante el conjunto de visitados.
--------------------------------	--

Campo	Descripción
N°	P-04
Objetivo	Corregir la lógica de gestión del conjunto de cerrados (visited) en el algoritmo A* para garantizar su optimalidad matemática y evitar el descarte de rutas óptimas.
Herramienta	Claude Sonnet5
Texto del prompt	Actúa como un Ingeniero de Software experto en Java e Inteligencia Artificial. En mi clase SearchAlgorithms.java, tengo un error conceptual crítico en el método de búsqueda informada aStar que rompe la garantía de optimalidad teórica. Estoy agregando estados al 'visited' al momento de generarlos y no al extraerlos. Por favor, refactoriza el método aStar aplicando la secuencia estricta: Extraer con poll() -> verificar si está en visited -> Agregar a visited -> Verificar meta -> Generar sucesores.
Resultado / cómo se usó	Se obtuvo la versión refactorizada del método aStar con la lógica correcta de closed set. El método greedy se mantuvo intencionalmente con su lógica de marcado al generar sucesores; debido a que Greedy no ofrece garantías de optimalidad por definición, esta decisión de diseño reduce significativamente el uso de memoria al impedir la acumulación de nodos redundantes en la cola de prioridad.
Validación / ajuste del equipo	Se sometió el código a una revisión teórica para asegurar que A* cumpla con las propiedades matemáticas de optimalidad en grafos enseñadas en el curso. Al recompilar el bloque Java y ejecutar los casos de prueba, se verificó que el algoritmo explora el árbol de estado correctamente sin violar la garantía de la ruta más corta, asegurando una defensa sólida en la sustentación.

3. Informe 2

Campo	Descripción
N°	P-05
Objetivo	Implementar el modelado del tablero de Othello (8×8), inicialización de fichas centrales y validación/captura bidireccional en 8 direcciones en Java.
Herramienta	Gemini 3.5 flash (high) (Antigravity).
Texto del prompt	Actúa como un programador experto en Java. Necesito implementar la clase Tablero.java para un juego de mesa de Othello (Reversi) de 8x8. La clase debe contener: 1. Una matriz 'matriz' de caracteres (8x8) donde '.' representa casilla vacía, 'X' representa fichas Negras y 'O' fichas Blancas. 2. Un constructor que inicialice el tablero con la configuración clásica de 4 fichas en el centro (2 blancas en diagonal y 2 negras en diagonal). 3. Un método para comprobar límites del tablero. 4. Un método que verifique si una coordenada es una jugada legal. Recuerda que en Othello una jugada es válida si y solo si encierra horizontal, vertical o diagonalmente una o más fichas rivales entre la nueva ficha y una propia existente. 5. Un método que retorne una lista de coordenadas válidas [fila, columna] para el turno activo. 6. Un método que juegue la ficha, realice los volteos correspondientes en las direcciones que correspondan y retorne true si la jugada fue exitosa. 7. Un método para clonación profunda del tablero para la simulación del Minimax. No uses 'package' en el código para facilitar la compilación por terminal.
Resultado / cómo se usó	Se generó la estructura de Board.java con las reglas clásicas de Othello, la cual se utilizó como el modelo central del estado del juego.
Validación / ajuste del equipo	Se probó manualmente que la captura en direcciones complejas (como diagonales y flanqueos múltiples) funcionara de forma correcta, detectando que la matriz de clones copiaba bien las posiciones sin alterar la de origen.

Campo	Descripción
N°	P-06
Objetivo	Diseñar el renderizado visual de consola con colores ANSI y la transición animada de volteo de fichas en tiempo real.

Herramienta	Gemini 3.5 flash (high) (Antigravity).
Texto del prompt	Quiero añadirle un aspecto visual premium a Tablero.java en la consola. Escribe el código necesario para: 1. Definir constantes ANSI para colores: fondo verde (\u001B[42m), bordes en gris oscuro (\u001B[90m) para que sean sutiles, y textos para leyendas. 2. Implementar un método que imprima el tablero: cada celda mide 3 caracteres, las celdas vacías son espacio, las fichas Negras (■) son en texto negro (\u001B[30m), las fichas Blancas (□) son en texto blanco brillante (\u001B[97m), las jugadas recomendadas (pistas) se muestran como un signo más amarillo '+' en la celda verde. En el pie de página ("Fichas ->") imprime el score usando etiquetas configurables (etiquetaNegro y etiquetaBlanco) con bloque gris para Negras y blanco para Blancas, sin fondo verde para que no choque si la consola tiene fondo negro. 3. Implementar un método para poder cambiar los nombres de Negras/Blancas dinámicamente según el modo de juego. 4. Implementar un método que haga una animación en 3 pasos: - Frame 1: Coloca la nueva ficha, limpia la pantalla con códigos ANSI (\033[H\033[2J) y renderiza (espera 450ms). - Frame 2: Coloca el caracter de transición 'I' (representado como un bloque difuminado '░░░░░' en amarillo brillante \u001B[33m) en todas las fichas capturadas, limpia pantalla y renderiza (espera 450ms). - Frame 3: Pone el color definitivo del jugador en las fichas capturadas, limpia pantalla y renderiza el final. Asegura que el constructor de clonación (Tablero copy constructor) y getClon() mantengan y copien los valores de etiquetaNegro y etiquetaBlanco.
Resultado / cómo se usó	Proporcionó los métodos de dibujo estilizado ANSI y la lógica de animación, logrando el efecto visual del giro físico de las fichas.
Validación / ajuste del equipo	Ajustamos la pausa del hilo a 450ms para que la consola muestre el volteo con una velocidad fluida pero discernible al ojo humano.

Campo	Descripción
Nº	P-07

Objetivo	Crear la función heurística en Java para Othello ponderando pesos de celdas, movilidad y paridad, con una optimización dinámica para las casillas adyacentes a las esquinas.
Herramienta	Gemini 3.5 flash (high) (Antigravity).
Texto del prompt	Implementa EvaluadorHeuristico.java para valorar estados del tablero. - Define una matriz de pesos estáticos PESOS_CASILLAS de 8x8 (Esquinas: +100; casillas C y X adyacentes a esquinas: -20 y -50 respectivamente; bordes: +10; centro: +1). - Implementa un método para obtener el peso de la casilla: si la celda tiene un peso de -20 o -50 (casillas C y X) pero la esquina correspondiente a su cuadrante (A1, H1, A8 o H8) ya no está vacía (tiene una ficha), anula la penalización y devuelve +5. Jugar cerca de una esquina tomada es seguro y consolida el flanco. En caso contrario, retorna el peso estático original. - En el método que evalúa el puntaje: - Si es terminal, devuelve +1,000,000 + diferencia si gana jugadorIA, -1,000,000 - diferencia si pierde, o 0 si empata. - Suma las puntuaciones de posición de cada jugador usando getPesoCasilla. - Calcula la movilidad: cantidad de jugadas válidas de jugadorIA menos las de jugadorHumano. - Calcula la paridad de fichas totales de jugadorIA menos las de jugadorHumano. - Combina los scores: (puntajePosicion * 10) + (puntajeMovilidad * 15) + (puntajeParidad * 2).
Resultado / cómo se usó	Generó una evaluación dinámica estratégica de alta calidad que hace a la IA esquivar las adyacencias de las esquinas al principio pero consolidarlas tácticamente al final.
Validación / ajuste del equipo	Comprobamos mediante pruebas en modo Alfa-Beta que esta heurística dinámica mejora drásticamente el desempeño de la IA de profundidad media, ganando esquinas de forma natural.

Campo	Descripción
N°	P-08
Objetivo	Codificar el motor de búsqueda Minimax recursivo con Poda Alfa-Beta tradicional y contadores de nodos evaluados, parametrizando el color de ficha activo para habilitar búsquedas independientes en el modo Espectador (IA vs. IA).

Herramienta	Gemini 3.5 flash (high) (Antigravity).
Texto del prompt	Escribe la clase AgenteMinimax.java en Java. Debe contar con: 1. Contadores nodosEvaluadosConPoda y nodosEvaluadosSinPoda, con métodos reiniciarContadores() para reiniciarlos. 2. Un método que encuentre la mejor jugada para el jugador. Inicializa el mejorMovimiento por seguridad con el primer elemento de tablero.getMovimientosLegales(jugador) (si no está vacío) para evitar retornos nulos ante máxima penalización. Usa recursión minimax para obtener la puntuación de cada hijo. 3. El método minimax recursivo: - Caso base: profundidad == 0 o tablero.esTerminal() -> retorna EvaluadorHeuristico.evaluar para el jugadorMaximizador. - Si el jugador activo no tiene jugadas válidas (Pass), salta el turno y llama recursivamente a minimax con (!esMaximizador), reduciendo la profundidad en 1. - Implementa las ramas de maximizador y minimizador actualizando alfa y beta. Si usarPoda es true, ejecuta la poda tradicional (si beta <= alfa, romper bucle).
Resultado / cómo se usó	Proporcionó el motor de toma de decisiones parametrizado. Esto permite que en el modo Espectador (IA vs. IA) cada agente calcule su movimiento optimizando para su respectivo color (IA 1 maximizando para Negras e IA 2 para Blancas), evitando que jueguen con el mismo color.
Validación / ajuste del equipo	Validamos la eficacia de la poda y la independencia de color: al enfrentar a IA 1 (Negras, Prof. 3) contra IA 2 (Blancas, Prof. 5) en modo espectador, cada una jugó en su turno buscando su propio beneficio de color, reportando métricas de nodos de forma independiente.

Campo	Descripción
Nº	P-09
Objetivo	Estructurar el bucle principal de Main.java, el menú rejugable de selección de modos (incluyendo el modo espectador IA vs. IA), y anclar la visualización del tablero en la parte superior.
Herramienta	Gemini 3.5 flash (high) (Antigravity).

Texto del prompt	Diseña la clase controladora Main.java en Java para Othello con las siguientes pautas: 1. Encierra la ejecución en un bucle while(true) infinito. Lo primero que debe hacer es limpiar la pantalla (\033[H\033[2J) y mostrar el menú: 1) Humano vs IA, 2) IA vs IA (Espectador), 3) Salir. 2. Si elige 1: pide seleccionar dificultad (profundidad 3, 5, 6 o personalizada). Negras es Humano, Blancas es IA. 3. Si elige 2: pide profundidad para IA 1 (Negras), profundidad para IA 2 (Blancas), y control de avance (1: Manual por ENTER, 2: Automático continuo con retardo de 1.8 segundos). Llama a tablero.setEtiquetasJugadores para configurar las etiquetas en modo espectador. 4. En el bucle de juego (!tablero.esTerminal()): - ANCLAJE DEL TABLERO: En cada refresco, limpia consola y dibuja de inmediato el tablero (imprimirTablero) en la parte superior. Las cabeceras de turno, coordenadas jugadas, tablas de métricas de poda y prompts se imprimen siempre debajo del tablero. - Turno Negras (Humano o IA 1): si es humano, valida coordenadas y juega con animación (pausa 1.2s). Si es IA 1, calcula jugada, anuncia coordenada de decisión, pausa 1.2s, anima la jugada, limpia pantalla, dibuja el tablero y muestra métricas. - Turno Blancas (IA o IA 2): ejecuta su turno de la misma forma que IA 1. 5. Al acabar la partida, muestra el tablero final arriba, el score con etiquetas correctas abajo y el ganador. Pide presionar ENTER para regresar al menú inicial.
Resultado / cómo se usó	Estructuró el flujo de turnos interactivo (gestionando tanto el modo Humano vs. IA como el Espectador IA vs. IA), implementando el anclaje visual (tablero fijo arriba) y la rejugabilidad mediante menús.
Validación / ajuste del equipo	Se corroboró que al terminar un duelo de IA vs. IA (o Humano vs. IA) se regrese correctamente al menú principal al presionar ENTER. También se validó que en modo automático las IAs jueguen fluidamente con el retardo de 1.8 segundos entre turnos.

4. Informe 3

Campo	Descripción
N°	P-10
Objetivo	Desarrollar el Informe 3 completo: generar el notebook ejecutable con el código del EDA, el preprocesamiento y los modelos de regresión.

Herramienta	Claude Code — Claude Fable 5 (Anthropic)
Texto del prompt	Actúa como un Data Scientist Senior. Necesito estructurar un Jupyter Notebook ejecutable en Python para la fase de análisis exploratorio (EDA) y modelado predictivo del caso NovaConecta. El notebook debe contener: Un bloque en Markdown con el contexto de negocio. Un bloque de carga para el archivo dataset_novaconecta.csv que implemente manejo de excepciones (try-except) para soportar ejecución tanto en entorno local como en Google Colab sin modificar el código. Un análisis exploratorio (EDA) que ejecute detección de nulos, identificación de outliers mediante la regla de 1.5x IQR, gráficos de distribución y matriz de correlaciones. Un flujo de preprocesamiento estricto encapsulado en un Pipeline de scikit-learn que aplique imputación por la mediana, winsorización y One-Hot Encoding. Una división de datos (split 80/20, random_state=42) donde la estandarización (StandardScaler) se ajuste exclusivamente sobre el conjunto de entrenamiento para evitar cualquier fuga de datos (data leakage). La instanciación y entrenamiento de los siguientes modelos: Regresión Lineal (Simple y Múltiple), Regresión Polinómica (con búsqueda del grado óptimo por validación cruzada), Ridge, Lasso y Gradient Boosting. Un bloque de evaluación que calcule MSE, RMSE y R^2 mediante validación cruzada de 5-folds, consolide las métricas en un DataFrame comparativo y grafique los residuos del mejor modelo. Asegúrate de que el código sea modular
Resultado / cómo se usó	La IA generó el notebook Informe_3_Regresion.ipynb y lo ejecutó completo sin errores: 25 nulos imputados, 8 outliers winsorizados, mejor modelo Regresión Polinómica grado 2.
Validación / ajuste del equipo	Verificado con ejecución completa tipo "Restart & Run All". Observación del QA atendida: el prompt solicita explícitamente la generación del código del notebook.

5. Informe 4

Campo	Descripción
Nº	P-11

Objetivo	Auditar técnicamente el dataset de NovaConecta antes de desarrollar el Informe 4, verificando calidad, construcción, variable objetivo, fuga de información, variables predictoras, preprocesamiento, balanceo, validación, modelos y estructura del notebook.
Herramienta	ChatGPT — GPT-5.6 Thinking
Texto del prompt	Actúa como un ingeniero senior de Machine Learning especializado en modelos de clasificación y predicción de abandono de clientes. Estoy desarrollando el Informe 4 de Clasificación del proyecto de Inteligencia Artificial de la empresa ficticia NovaConecta. El repositorio contiene el dataset <code>bloque_b_python/data/dataset_novaconecta.csv</code> y el script <code>bloque_b_python/data/generar_dataset.py</code>. Realiza una auditoría técnica completa del dataset antes de comenzar el notebook. Necesito que: 1. Verifiques sus dimensiones, columnas, tipos de datos, valores faltantes, duplicados, valores atípicos y distribución de la variable abandono. 2. Analices cómo fue construida cada variable en <code>generar_dataset.py</code>. 3. Identifiques posibles casos de data leakage o fuga de información. 4. Definas cuál será la variable objetivo y cómo debe codificarse. 5. Indiques qué variables deben utilizarse, cuáles deben excluirse y justifiques cada decisión. 6. Definas una estrategia correcta de división train/test estratificada. 7. Propongas el preprocesamiento para variables numéricas y categóricas. 8. Analices si el desbalance de clases requiere SMOTE, <code>class_weight</code> o ninguna técnica, explicando cómo se comprobará. 9. Propongas los modelos requeridos por la rúbrica: Regresión Logística y al menos tres entre KNN, SVC, Naive Bayes, Decision Tree y Random Forest, además de un modelo de ensamble o boosting. 10. Definas las métricas, la validación cruzada y la estrategia de ajuste de hiperparámetros. 11. Diseñas la estructura completa del notebook <code>Informe_4_Clasificacion.ipynb</code>. No inventes métricas ni resultados que todavía no hayan sido ejecutados. Señala claramente cualquier riesgo técnico encontrado.

Resultado / cómo se usó	Se revisó el dataset y el script generador. Se confirmó que tiene 1200 registros, 25 datos faltantes y un desbalance moderado. También se detectó que `valor_vida_cliente` usa información de `abandono`, por lo que se excluyó para evitar fuga de datos.
Validación / ajuste del equipo	Se comprobaron los datos directamente en el CSV y en el script. Se eliminaron del modelo las columnas que no aportaban información o causaban fuga de datos.

Campo	Descripción
N°	P-12
Objetivo	Crear la primera parte del notebook con contexto, carga reproducible, EDA orientado a clasificación, selección de variables, split estratificado y pipeline de preprocesamiento sin entrenar modelos.
Herramienta	ChatGPT — GPT-5.6 Thinking
Texto del prompt	Continúa el Informe 4 de Clasificación de NovaConecta utilizando las decisiones aprobadas en la auditoría técnica anterior y lo que hay en el nuevo repositorio. Crea el archivo bloque_b_python/notebooks/Informe_4_Clasificacion.ipynb y desarrolla la primera parte del notebook de manera profesional, documentada y ejecutable en Visual Studio Code y Google Colab. El notebook debe incluir: 1. Encabezado con universidad, curso, empresa ficticia, integrantes, responsable David Joel Aldana Chavez y objetivo. 2. Descripción del problema de abandono de clientes de NovaConecta. 3. Importación ordenada de librerías. 4. Configuración reproducible con semilla 42. 5. Una forma robusta de localizar dataset_novaconecta.csv mediante rutas relativas, independientemente de si el notebook se ejecuta desde la raíz o desde la carpeta notebooks. 6. Carga del dataset. 7. Dimensiones, tipos, resumen estadístico, duplicados y valores faltantes. 8. Gráficos relevantes para la variable abandono y las principales variables predictoras.

	9. Análisis del desbalance de clases. 10. Explicación técnica de por qué valor_vida_cliente produce data leakage. 11. Selección justificada de X e y. 12. Codificación de abandono como No=0 y Si=1. 13. División train/test con test_size=0.20, random_state=42 y stratify=y. 14. Identificación separada de variables numéricas y categóricas. 15. Pipeline con SimpleImputer, OneHotEncoder y el escalador más apropiado. 16. ColumnTransformer para encapsular el preprocesamiento. 17. Verificaciones de que el conjunto de prueba no participa en el ajuste. No apliques SMOTE todavía y no entrenes los modelos finales en esta etapa. Utiliza celdas Markdown en español para explicar cada decisión. Evita código duplicado y funciones innecesariamente complejas. Ejecuta todas las celdas creadas y comprueba que no existan errores.
Resultado / cómo se usó	Se creó la primera parte del notebook con la carga de datos, EDA, selección de variables, división 80/20 y preprocesamiento con imputación, One-Hot Encoding y StandardScaler.
Validación / ajuste del equipo	Se ejecutó todo sin errores. Se verificó que el preprocesamiento aprendiera solo con los datos de entrenamiento y que el conjunto de prueba quedara separado.

Campo	Descripción
N°	P-13
Objetivo	Implementar y comparar seis modelos base mediante pipelines, validación cruzada estratificada y métricas orientadas a detectar abandono.
Herramienta	ChatGPT — GPT-5.6 Thinking
Texto del prompt	Continúa el notebook bloque_b_python/notebooks/Informe_4_Clasificacion.ipynb de NovaConecta. Implementa una etapa de modelado base utilizando exactamente el mismo train/test y el ColumnTransformer creado anteriormente.

	Entrena estos modelos: 1. Regresión Logística. 2. K-Nearest Neighbors. 3. Support Vector Classifier. 4. Árbol de Decisión. 5. Random Forest. 6. AdaBoost o Gradient Boosting como modelo de boosting. Requisitos técnicos: - Cada modelo debe estar encapsulado en un Pipeline. - Mantén random_state=42 en los modelos que lo permitan. - No utilices SMOTE todavía. - No ajustes hiperparámetros todavía. - Utiliza StratifiedKFold de 5 particiones. - Calcula resultados de validación cruzada solamente con el conjunto de entrenamiento. - Evalúa en el conjunto de prueba Accuracy, Precision, Recall, F1-Score y ROC-AUC cuando el modelo lo permita. - Usa zero_division=0 en las métricas cuando corresponda. - Registra el tiempo de entrenamiento de cada modelo. - Construye una tabla comparativa ordenada. - No selecciones un modelo basándote únicamente en Accuracy. - Analiza especialmente el Recall y F1 de la clase abandono. - Explica las diferencias entre modelos lineales, basados en distancia, margen, árboles y boosting. - Guarda opcionalmente la tabla como bloque_b_python/resultados/informe_4/metricas_modelos_base.csv. No utilices el conjunto de prueba para modificar los hiperparámetros. Las conclusiones de esta etapa deben considerarse preliminares. Ejecuta las nuevas celdas y verifica que el notebook funcione de principio a fin.
Resultado / cómo se usó	Se entrenaron seis modelos base: Regresión Logística, KNN, SVC, Árbol de Decisión, Random Forest y AdaBoost. La Regresión Logística obtuvo el mejor F1 de validación cruzada: 0.6755.

Validación / ajuste del equipo	Se comprobó que todos los modelos usaran el mismo entrenamiento, prueba y pipeline. No se aplicó balanceo ni ajuste de parámetros en esta etapa.
--------------------------------	---

Campo	Descripción
N°	P-14
Objetivo	Comparar sin balanceo, <code>class_weight='balanced'</code> y SMOTE, garantizando que el balanceo ocurra únicamente dentro del entrenamiento y de cada fold.
Herramienta	ChatGPT — GPT-5.6 Thinking
Texto del prompt	Estoy analizando si el desbalance de la variable `abandono` afecta el rendimiento de los modelos. Quiero comparar tres alternativas: trabajar con las clases originales, usar <code>class_weight='balanced'</code> y aplicar SMOTE. ¿Cómo debería realizar esta comparación con Regresión Logística, SVC y Random Forest usando <code>StratifiedKFold`</code> de 5 particiones?</p> <p>SMOTE debe usar <code>random_state=42`</code> y ejecutarse dentro de un pipeline de <code>imbalanced-learn`</code>, después del preprocesamiento y solamente sobre el entrenamiento de cada fold. Oriéntame sobre cómo comparar Precision, Recall, F1 y ROC-AUC, y cómo interpretar el costo de falsos positivos y falsos negativos para decidir qué estrategia conviene conservar."Continúa el Informe 4 de Clasificación de NovaConecta y analiza rigurosamente el desbalance de la variable abandono. No asumas que SMOTE es automáticamente beneficioso. Compara al menos estas estrategias: 1. Sin balanceo. 2. <code>class_weight='balanced'</code> en los modelos que lo soporten. 3. SMOTE aplicado correctamente dentro de un Pipeline de <code>imbalanced-learn</code>. Requisitos:

	- Utiliza únicamente los datos de entrenamiento para las comparaciones. - Aplica SMOTE después del preprocesamiento y nunca antes de train_test_split. - No apliques SMOTE al conjunto de prueba. - Asegúrate de que SMOTE se ejecute dentro de cada partición de validación cruzada para evitar data leakage. - Utiliza StratifiedKFold con 5 particiones. - Compara F1, Recall, Precision y ROC-AUC. - Muestra la distribución de clases original y la distribución generada por SMOTE en el entrenamiento. - Evalúa las estrategias con los modelos más apropiados, como Regresión Logística, SVC y Random Forest. - Analiza el costo de falsos positivos y falsos negativos en el contexto de NovaConecta. - Determina qué estrategia de balanceo se utilizará para el ajuste de hiperparámetros. - Justifica la decisión con resultados reales y no solamente con teoría. - Actualiza la tabla comparativa del notebook. Ejecuta todo el notebook para verificar que los pipelines sean compatibles y que no exista fuga de información. Al finalizar proporciona: - Tabla comparativa de estrategias. - Decisión final sobre balanceo. - Justificación de negocio. - Cambios introducidos en el notebook. Requisito adicional de reproducibilidad: - Instancia SMOTE con `random_state=42` o `random_state=SEED` en todos los pipelines y demostraciones para obtener resultados repetibles."
Resultado / cómo se usó	Se compararon tres estrategias: sin balanceo, `class_weight` y SMOTE. SMOTE obtuvo el mejor F1 y Recall promedio, por lo que se eligió para la optimización.
Validación / ajuste del equipo	Se verificó que SMOTE se aplicara solo dentro del entrenamiento y de cada fold. El conjunto de prueba no fue balanceado ni usado para elegir la estrategia.

Campo	Descripción
N°	P-15
Objetivo	Optimizar los tres modelos más prometedores mediante GridSearchCV, seleccionar el ganador por validación cruzada y evaluar los modelos optimizados sin usar el test durante la búsqueda.
Herramienta	ChatGPT — GPT-5.6 Thinking
Texto del prompt	Continúa el notebook Informe_4_Clasificacion.ipynb utilizando los resultados reales de los modelos base y del análisis de balanceo. Selecciona los tres modelos más prometedores según la validación cruzada del conjunto de entrenamiento. Entre ellos debe existir al menos un modelo de ensamble o boosting. Realiza ajuste de hiperparámetros utilizando GridSearchCV o RandomizedSearchCV. Requisitos: 1. El ajuste debe realizarse sobre pipelines completos que incluyan preprocesamiento y la estrategia de balanceo elegida. 2. Utiliza StratifiedKFold de 5 particiones. 3. Mantén el conjunto de prueba completamente separado durante la búsqueda. 4. Define espacios de hiperparámetros razonables y justificados. 5. Utiliza F1 como métrica principal de optimización, considerando también Recall, Precision, Accuracy y ROC-AUC. 6. Evita búsquedas excesivamente grandes que no aporten valor. 7. Muestra los mejores parámetros y la mejor puntuación de validación cruzada. 8. Compara cada modelo optimizado con su versión base. 9. Evalúa una sola vez los modelos optimizados en el conjunto de prueba. 10. Construye la tabla comparativa definitiva. 11. Selecciona el mejor modelo considerando rendimiento, estabilidad, costo de errores e interpretabilidad. 12. Guarda opcionalmente: - metricas_modelos_finales.csv - mejores_parametros.json - mejor_modelo_clasificacion.joblib

	No selecciones el modelo basándote repetidamente en el conjunto de prueba. No inventes resultados y reporta cualquier señal de sobreajuste. Al finalizar proporciona: - Mejores parámetros reales. - Resultados de validación cruzada. - Resultados finales del test. - Justificación del modelo seleccionado. - Lista de archivos creados o modificados. Utiliza estos espacios de búsqueda controlados: - Regresión Logística: <code>`C=[0.01, 0.1, 1, 10]`</code> y <code>`solver=['liblinear', 'lbfgs']`</code>. - SVC: <code>`C=[0.1, 1, 10, 30]`</code>, <code>`gamma=['scale', 0.01, 0.1]`</code> y <code>`kernel=['rbf']`</code>. - Random Forest: <code>`n_estimators=[100, 200]`</code>, <code>`max_depth=[5, 10]`</code>, <code>`min_samples_leaf=[2]`</code> y <code>`max_features=['sqrt']`</code>. No amplíes estos rangos sin una justificación técnica.
Resultado / cómo se usó	Se optimizaron Regresión Logística, SVC y Random Forest con GridSearchCV. SVC obtuvo el mejor F1 de validación cruzada: 0.7006, con <code>`C=1`</code> , <code>`gamma=0.01`</code> y kernel RBF.
Validación / ajuste del equipo	Se comprobó que GridSearchCV utilizara solo los datos de entrenamiento. El modelo ganador se eligió por validación cruzada, no por los resultados del test.

Campo	Descripción
N°	P-16
Objetivo	Completar la evaluación final, interpretabilidad, conclusiones y auditoría de calidad del notebook, verificando el cumplimiento integral de la rúbrica.
Herramienta	ChatGPT — GPT-5.6 Thinking
Texto del prompt	Realiza la revisión final del notebook bloque_b_python/notebooks/Informe_4_Clasificacion.ipynb de NovaConecta como ingeniero senior de Machine Learning y auditor de calidad.

	Completa y verifica: 1. Matriz de confusión del mejor modelo. 2. Classification report. 3. Accuracy, Precision, Recall y F1-Score. 4. ROC-AUC y curva ROC, si corresponde. 5. Interpretación detallada de verdaderos positivos, falsos positivos, falsos negativos y verdaderos negativos. 6. Análisis del costo empresarial de los falsos negativos. 7. Importancia de variables mediante coeficientes, feature_importances_ o permutation importance, según el modelo elegido. 8. Interpretación de las variables más influyentes. 9. Comparación final de todos los modelos. 10. Justificación del modelo ganador. 11. Recomendaciones concretas para que NovaConecta reduzca el abandono. 12. Limitaciones del dataset sintético y del modelo. 13. Conclusiones técnicas y de negocio. 14. Información de versiones, semilla y reproducibilidad. 15. Instrucciones breves para ejecutar el notebook. Después realiza una auditoría completa: - Comprueba que no se utilice valor_vida_cliente como predictor. - Comprueba que no exista data leakage. - Comprueba que train y test se separen antes del preprocesamiento. - Comprueba que SMOTE se aplique solamente al entrenamiento. - Comprueba que GridSearchCV no utilice el test. - Comprueba que todas las rutas sean relativas. - Comprueba que todas las celdas se ejecuten en orden. - Elimina código duplicado, variables sin usar y salidas innecesarias. - Ejecuta Restart Kernel and Run All. - Verifica el cumplimiento de cada criterio de la rúbrica del Informe 4. No inventes métricas, cambios ni validaciones que no se hayan realizado.
--	---

	Requisito de interpretabilidad: - Calcula `permutation_importance` con `scoring='f1'`, `random_state=42` o `random_state=SEED`, para que la importancia de variables sea coherente con la métrica usada para seleccionar el modelo.
Resultado / cómo se usó	Se completó la evaluación final con matriz de confusión, métricas, curva ROC, importancia de variables, conclusiones y recomendaciones. El SVC obtuvo F1 de 0.6567 y Recall de 0.7174 en test.
Validación / ajuste del equipo	Se ejecutó todo el notebook desde cero y sin errores. Se verificó que no hubiera fuga de datos, que SMOTE y GridSearchCV usaran solo entrenamiento y que se cumpliera la rúbrica.

6. Informe 5

Campo	Descripción
N°	P-17
Objetivo	Seleccionar las variables, definir la estrategia de escalamiento y preparar el preprocesamiento de datos para los algoritmos espaciales.
Herramienta	Gemini 3.1 Pro (High) (Antigravity)
Texto del prompt	Actúa como un ingeniero senior de Machine Learning. Estoy desarrollando el Informe 5 de Agrupamiento (Clustering) no supervisado para la empresa NovaConecta. Analiza las 13 variables del dataset y selecciona justificadamente cuáles deben usarse para el agrupamiento. Excluye identificadores, la variable objetivo (abandono) y variables categóricas no ordinales para evitar distorsiones. Propón la mejor técnica de escalamiento considerando que variables monetarias como valor_vida_cliente tienen rangos numéricos gigantes frente a satisfaccion_cliente (1-10). Diseña el código inicial para cargar los datos y aplicar este preprocesamiento de forma limpia.
Resultado / cómo se usó	El modelo seleccionó 7 variables numéricas enfocadas en comportamiento y valor económico. Se excluyeron los identificadores y variables de texto. Propuso usar

	StandardScaler para que todas las variables tengan media 0 y varianza 1, evitando que las variables monetarias dominen el cálculo de distancias euclidianas.
Validación / ajuste del equipo	Se integró el código al notebook. El equipo validó las estadísticas descriptivas post-escalamiento mediante .describe(), verificando empíricamente que las desviaciones estándar quedaron en ~1, confirmando que la transformación fue exitosa.

Campo	Descripción
N°	P-18
Objetivo	Implementar K-Means, determinar matemáticamente el número óptimo de clústeres (K) y asegurar reproducibilidad estricta.
Herramienta	Gemini 3.1 Pro (High) (Antigravity)
Texto del prompt	Continúa el Informe 5. Implementa el algoritmo K-Means sobre la data escalada. Evalúa el rango de K=2 hasta K=10. Genera el código para calcular y graficar en paralelo tres métricas: el Método del Codo (Inercia), el Coeficiente de Silueta y el Índice Davies-Bouldin. Luego, instancia el modelo final usando el mejor K encontrado, asegurándote de usar una semilla de aleatoriedad fija (random_state=42 o random_state=SEED) al instanciar KMeans para garantizar la reproducibilidad de los centroides, y genera un gráfico del perfil de Silueta por clúster.
Resultado / cómo se usó	Se generó el bucle de iteración. Las métricas indicaron que K=2 maximizaba la silueta y generaba el codo más pronunciado. Se instanció el modelo final con random_state=SEED, asegurando que el algoritmo converja exactamente a los mismos grupos en cada ejecución.
Validación / ajuste del equipo	El equipo analizó visualmente las gráficas, fijó K_OPTIMO=2 y validó que, gracias a la instrucción explícita de la semilla, la reproducibilidad estaba garantizada y la asignación de clientes no variaba entre diferentes corridas del notebook.

Campo	Descripción
N°	P-19
Objetivo	Implementar DBSCAN optimizando sus hiperparámetros de manera exhaustiva y compararlo técnicamente con K-Means.
Herramienta	Gemini 3.1 Pro (High) (Antigravity)
Texto del prompt	Ahora implementa el algoritmo DBSCAN. Realiza lo siguiente: 1. Genera una curva de k-distancias (NearestNeighbors) para estimar visualmente eps. 2. Implementa un grid search manual probando eps (0.3 a 1.6) y min_samples (5 a 15). 3. Selecciona la mejor combinación que maximice Silueta minimizando el ruido. 4. Aplica un Análisis de Componentes Principales (PCA) con 2 componentes para reducir la dimensionalidad y proyectar los clústeres de K-Means y DBSCAN en un gráfico de dispersión bidimensional. 5. Crea una tabla comparativa final (K-Means vs DBSCAN).
Resultado / cómo se usó	Se calculó la curva de k-distancias (codo ~1.5) y se eligió eps=1.4 y min_samples=5. El LLM generó exitosamente el código PCA para comprimir las 7 dimensiones a 2, permitiendo mapear gráficamente el espacio hiperdimensional de ambos modelos.
Validación / ajuste del equipo	Se validaron los gráficos de dispersión PCA. Se corroboró visualmente en el plano 2D que DBSCAN agrupó a la gran mayoría en un solo clúster masivo, aislando 34 puntos como ruido, lo que validó la decisión gerencial de utilizar K-Means.

7. Informe 6

Campo	Descripción
N°	P-20
Objetivo	Implementar Análisis de Componentes Principales (PCA) para reducir la dimensionalidad de las 7 variables numéricas, escalar los datos y seleccionar el número óptimo de componentes.
Herramienta	Gemini 3.1 Pro (High) (Antigravity)
Texto del prompt	Actúa como un Científico de Datos senior. Estoy trabajando en el Informe 6 sobre Reducción de Dimensionalidad para el dataset de NovaConecta (7 variables numéricas). Genera el código en Python usando scikit-learn para escalar los datos con

	StandardScaler y aplicar un PCA completo. Luego, genera un Scree Plot y un gráfico de varianza acumulada para determinar cuántos componentes necesitamos para retener al menos el 80% de la información. Finalmente, extrae matemáticamente las cargas (loadings) de los componentes seleccionados y grafica un Biplot para proyectar los clientes.
Resultado / cómo se usó	Se generó el código base del pipeline. El modelo determinó que con 4 componentes se explica el 92.15% de la varianza. Se logró extraer la matriz de loadings y se trazó el Biplot en 2D que muestra los vectores de influencia.
Validación / ajuste del equipo	Se verificó exhaustivamente que el escalador (StandardScaler) se aplicara antes del PCA para evitar distorsiones. Se validó matemáticamente la matriz <code>pca.components_</code> para interpretar correctamente que el PC4 estaba dominado al 99% por la variable 'edad'. Se ajustaron manualmente los colores del Biplot para seguir la paleta del proyecto.

Campo	Descripción
N°	P-21
Objetivo	Aplicar t-SNE para visualización 2D no lineal y proyectar los clústeres generados en el informe previo (K-Means, K=2) en los nuevos espacios reducidos.
Herramienta	Gemini 3.1 Pro (High) (Antigravity)
Texto del prompt	Continúa con el notebook del informe 6. Ahora aplica t-SNE a los datos escalados para reducirlos a 2 dimensiones (para propósitos estrictamente de visualización), usando <code>max_iter=1000</code> , <code>random_state=42</code> e <code>init='pca'</code> para garantizar estabilidad, convergencia rápida y reproducibilidad. Crea gráficos de dispersión comparando PCA 2D y t-SNE 2D, coloreando los puntos primero por 'plan contratado' y luego por la variable 'abandono'. Después, reproduce el algoritmo K-Means con K=2 que determinamos en el Informe 5 y grafica esos clústeres tanto en el espacio PCA 2D como en t-SNE 2D, marcando los centroides en el PCA.
Resultado / cómo se usó	Se implementó t-SNE (<code>n_components=2</code>) usando los hiperparámetros robustos indicados y se generaron figuras. El resultado demostró que t-SNE agrupa visualmente mejor a los clientes con riesgo de abandono. Además, se logró superponer la etiqueta de K-Means (K=2) sobre el mapa topológico del t-SNE.

Validación / ajuste del equipo	Se fijaron explícitamente los hiperparámetros en la función t-SNE (evitando el deprecado n_iter) para asegurar la reproducibilidad del experimento en distintas máquinas sin lanzar warnings. Se comprobó que la distribución de K-Means coincidiera exactamente con el Informe 5 (Clúster 0: 394 clientes, Clúster 1: 806 clientes).
--------------------------------	---

Campo	Descripción
Nº	P-22
Objetivo	Comparar el rendimiento de un modelo predictivo (Regresión Logística) entrenado con las 7 variables originales versus el dataset reducido con PCA (4 componentes) y definir conclusiones de negocio.
Herramienta	Gemini 3.1 Pro (High) (Antigravity)
Texto del prompt	Para finalizar el Informe 6, necesitamos evaluar si el PCA sirve como preprocesamiento para predecir el abandono. Antes de entrenar, codifica explícitamente la variable objetivo 'abandono' mapeando 'No' a 0 y 'Si' a 1. Crea un script con dos pipelines de Regresión Logística: uno entrenado con las 7 variables escaladas originales, y otro que incluya PCA con 4 componentes antes de la regresión. Compara ambos modelos usando validación cruzada estratificada (cv=5) con la métrica F1-Score. Genera un boxplot para comparar el rendimiento. Luego, crea un DataFrame a modo de resumen comparativo (Espacio original vs PCA 4 vs t-SNE 2D).
Resultado / cómo se usó	La validación cruzada evidenció una severa caída del F1-Score al usar PCA (de 0.7922 a 0.6641). Se generó la tabla comparativa indicando dimensiones, varianza retenida y uso principal.
Validación / ajuste del equipo	El equipo verificó que la codificación manual de la variable objetivo garantizó métricas fiables. Se ejecutó el script y se rechazó la idea teórica de usar PCA como preprocesamiento final para el modelo de Churn. Se reescribieron las conclusiones de negocio para dejar en claro que no se debe usar PCA en el predictivo, pero rescatando el valor del PCA 2D y t-SNE para segmentación directiva.